

Perfil sociodemográfico de la paternidad: clasificación con regresión logística y regularización

Portafolio de Ciencia de Datos · Proyecto 2 · Implementación en Python

Ricardo José Córdova Soriano · Septiembre 2026

Objetivo y enfoque

Este proyecto construye un **modelo de clasificación que estima la probabilidad de que una persona tenga hijos** a partir de su perfil sociodemográfico: edad, estado civil, género, raza, clase social, empleo, ingresos, región y preferencias políticas. Los datos provienen de la *General Social Survey* (GSS) de Estados Unidos.

El análisis responde a tres preguntas:

1. **¿Qué variables explican realmente la paternidad?** Se aplican métodos automáticos de selección de variables (*stepwise*, *forward* y *backward*) con dos criterios de información (AIC y BIC).
2. **¿Qué modelo generaliza mejor?** Se comparan los candidatos con validación cruzada repetida (5 *folds* × 20 repeticiones) y se evalúan en un conjunto de prueba independiente.
3. **¿Se puede lograr un modelo más simple con igual capacidad predictiva?** Se aplican modelos regularizados (LASSO, Ridge y Elastic Net) y se ajustan sus hiperparámetros mediante validación cruzada.

Esta es la traducción a Python del análisis original en R. Los resultados coinciden numéricamente con los de R, incluidas todas las particiones aleatorias (ver el informe de verificación).

1. Configuración del entorno

Además de las librerías estándar, se usa el módulo `rcompat.py` del proyecto, que replica en Python las funciones de R sin equivalente directo: el generador aleatorio de R, las particiones de `caret`, la selección de variables de `step()` y los modelos regularizados de `glmnet`.

```
import math
import warnings

import numpy as np
import pandas as pd
import pyreadr                                # lectura de archivos .rds de R
import statsmodels.api as sm
import statsmodels.formula.api as smf
from scipy import stats
import matplotlib.pyplot as plt
import matplotlib.colors as mcolors
from matplotlib.lines import Line2D
from great_tables import GT, style, loc      # port de {gt} a Python
```

```

from IPython.display import display, Markdown

from rcompat import (RRandom, create_data_partition, caret_repeatedcv_folds, step_glm,
                     model_matrix, GlmLogitR, Glmnet, CVGlmnet, auc)

warnings.filterwarnings("ignore")

COL_PRINCIPAL, COL_ACENTO = "#1F4E79", "#C55A11"
PALETA_MODELOS = ["#8A99A6", "#1F4E79", "#C55A11", "#2E7D32"]

plt.rcParams.update({
    "figure.figsize": (8, 4.5), "figure.dpi": 110, "savefig.dpi": 150,
    "axes.spines.top": False, "axes.spines.right": False, "axes.spines.left": False,
    "axes.spines.bottom": False, "axes.grid": True, "grid.color": "#EBEBEB", "axes.axisbelow": True,
    "axes.titleweight": "bold", "axes.titlesize": 13, "axes.titlelocation": "left",
    "font.size": 11, "xtick.bottom": False, "ytick.left": False,
})

def titulos(fig_o_ax, titulo, subtítulo=None):
    """Título y subtítulo al estilo ggplot2 (sobre un eje o sobre la figura completa)."""
    if isinstance(fig_o_ax, plt.Figure):
        fig_o_ax.suptitle(titulo, x=0.02, y=0.995, ha="left", va="top", fontweight="bold", fontsize=13)
        if subtítulo:
            fig_o_ax.text(0.02, 0.945, subtítulo.replace("$", r"\$"), fontsize=10, color="#595959",
                          va="top")
    else:
        fig_o_ax.set_title(titulo, pad=22 if subtítulo else 8)
        if subtítulo:
            fig_o_ax.text(0, 1.02, subtítulo.replace("$", r"\$"), transform=fig_o_ax.transAxes,
                          fontsize=10, color="#595959")

def tabla(df, titulo, subtítulo=None, nota=None):
    """Tabla great_tables con estilo común."""
    t = (GT(df)
         .tab_header(title=titulo, subtitle=subtítulo)
         .opt_style(style=6, color="blue")
         .tab_options(table_font_size="13px", heading_title_font_size="16px"))
    if nota:
        t = t.tab_source_note(nota)
    return t

def fmt_p(p):
    return ["< 0.001" if v < 0.001 else f"{v:.4f}" for v in np.atleast_1d(p)]

def estrellas(p):
    return ["***" if v < 0.001 else "*" if v < 0.01 else "." if v < 0.05 else "" if v < 0.1 else ""
            for v in np.atleast_1d(p)]

def num(x, d=3):
    return f"{float(x):.{d}f}"

def pct(x, d=1):
    return f"{100 * float(x):.{d}f} %"

def kappa(obs, pred):
    """Índice Kappa de Cohen (como caret::confusionMatrix)."""
    obs, pred = np.asarray(obs, bool), np.asarray(pred, bool)
    po = np.mean(obs == pred)
    pe = np.mean(obs) * np.mean(pred) + np.mean(~obs) * np.mean(~pred)
    return (po - pe) / (1 - pe)

def metricas_clasif(prob, obs, umbral):
    """AUC, accuracy, Kappa, sensibilidad y especificidad (clase positiva: 'Si')."""
    obs = np.asarray(obs == "Si")
    pred = np.asarray(prob) >= umbral
    return {"AUC": auc(obs, pred), "Accuracy": np.mean(obs == pred), "Kappa": kappa(obs, pred),
            "Sensibilidad": np.mean(pred[obs]), "Especificidad": np.mean(~pred[~obs])}

```

```
def curva_roc(obs, prob):
    """Puntos (FPR, TPR) de la curva ROC, agrupando probabilidades empatadas."""
    obs = np.asarray(obs == "Si")
    df = pd.DataFrame({"p": prob, "y": obs}).groupby("p", sort=True)["y"].agg(["sum", "count"])[::-1]
    tp = np.concatenate([[0], df["sum"].cumsum().to_numpy()])
    fp = np.concatenate([[0], (df["count"] - df["sum"]).cumsum().to_numpy()])
    return fp / (~obs).sum(), tp / obs.sum()
```

2. Datos

```
datos = pyreadr.read_r("../datos/DatosGSS")[None]
datos.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 2035 entries, 0 to 2034
Data columns (total 13 columns):
#   Column          Non-Null Count  Dtype
---  ---
0   ClaseSocial     2035 non-null  category
1   Felicidad       2035 non-null  category
2   Politica        2035 non-null  category
3   Tamano          2035 non-null  float64
4   Region          2035 non-null  category
5   Ingreso         2035 non-null  category
6   Raza            2035 non-null  category
7   Genero          2035 non-null  category
8   Edad           2035 non-null  float64
9   Hijos           2035 non-null  category
10  EstadoCivil     2035 non-null  category
11  Empleo          2035 non-null  category
12  Zodiaco         2035 non-null  category
dtypes: category(11), float64(2)
memory usage: 54.5 KB
```

```
diccionario = pd.DataFrame({
    "Variable": datos.columns,
    "Tipo": [f"Categorica ({len(datos[c].cat.categories)} niveles)"
            if isinstance(datos[c].dtype, pd.CategoricalDtype) else "Numérica" for c in datos.columns],
    "Descripción": ["Clase social autopercibida",
                    "Nivel de felicidad declarado",
                    "Afiliación política",
                    "Tamaño de la localidad de residencia (miles de habitantes)",
                    "Región de Estados Unidos",
                    "Nivel de ingresos (codificado 0, 1, 2)",
                    "Raza",
                    "Género",
                    "Edad (años)",
                    "Tiene al menos un hijo – variable objetivo",
                    "Estado civil",
                    "Situación laboral",
                    "Signo del zodiaco (variable de control sin relación esperada)"]})
tabla(diccionario, "Diccionario de variables",
      f"{len(datos):,} personas encuestadas · {datos.shape[1]} variables · "
      f"{datos.isna().sum().sum()} valores faltantes")
```

Diccionario de variables

2,035 personas encuestadas · 13 variables · 0 valores faltantes

Variable	Tipo	Descripción
ClaseSocial	Categórica (3 niveles)	Clase social autopercebida
Felicidad	Categórica (3 niveles)	Nivel de felicidad declarado
Politica	Categórica (3 niveles)	Afiliación política
Tamano	Numérica	Tamaño de la localidad de residencia (miles de habitantes)
Region	Categórica (4 niveles)	Región de Estados Unidos
Ingreso	Categórica (3 niveles)	Nivel de ingresos (codificado 0, 1, 2)
Raza	Categórica (3 niveles)	Raza
Genero	Categórica (2 niveles)	Género
Edad	Numérica	Edad (años)
Hijos	Categórica (2 niveles)	Tiene al menos un hijo — variable objetivo
EstadoCivil	Categórica (3 niveles)	Estado civil
Empleo	Categórica (5 niveles)	Situación laboral
Zodiaco	Categórica (12 niveles)	Signo del zodiaco (variable de control sin relación esperada)

La variable objetivo **Hijos** se recodifica con etiquetas descriptivas (**No** / **Si**), igual que en la versión en R.

```
datos["Hijos"] = datos["Hijos"].cat.rename_categories({"0": "No", "1": "Si"})
conteo = datos["Hijos"].value_counts(sort=False)
(tabla(pd.DataFrame({"Hijos": conteo.index.astype(str), "Personas": conteo.values,
                    "Proporción": conteo.values / len(datos)}),
      "Distribución de la variable objetivo")
    .fmt_percent(columns="Proporción", decimals=1)
    .fmt_number(columns="Personas", decimals=0))
```

Distribución de la variable objetivo

Hijos	Personas	Proporción
No	567	27.9%
Si	1,468	72.1%

El 72.1 % de las personas encuestadas tiene hijos. La clase positiva es mayoritaria, así que un clasificador trivial que siempre prediga "Sí" acertaría en ese mismo porcentaje de casos. Por eso la exactitud (*accuracy*) no basta para evaluar los modelos, y se usan también el **AUC** y el **índice Kappa**, que descuentan el acierto atribuible al azar.

Análisis exploratorio

```

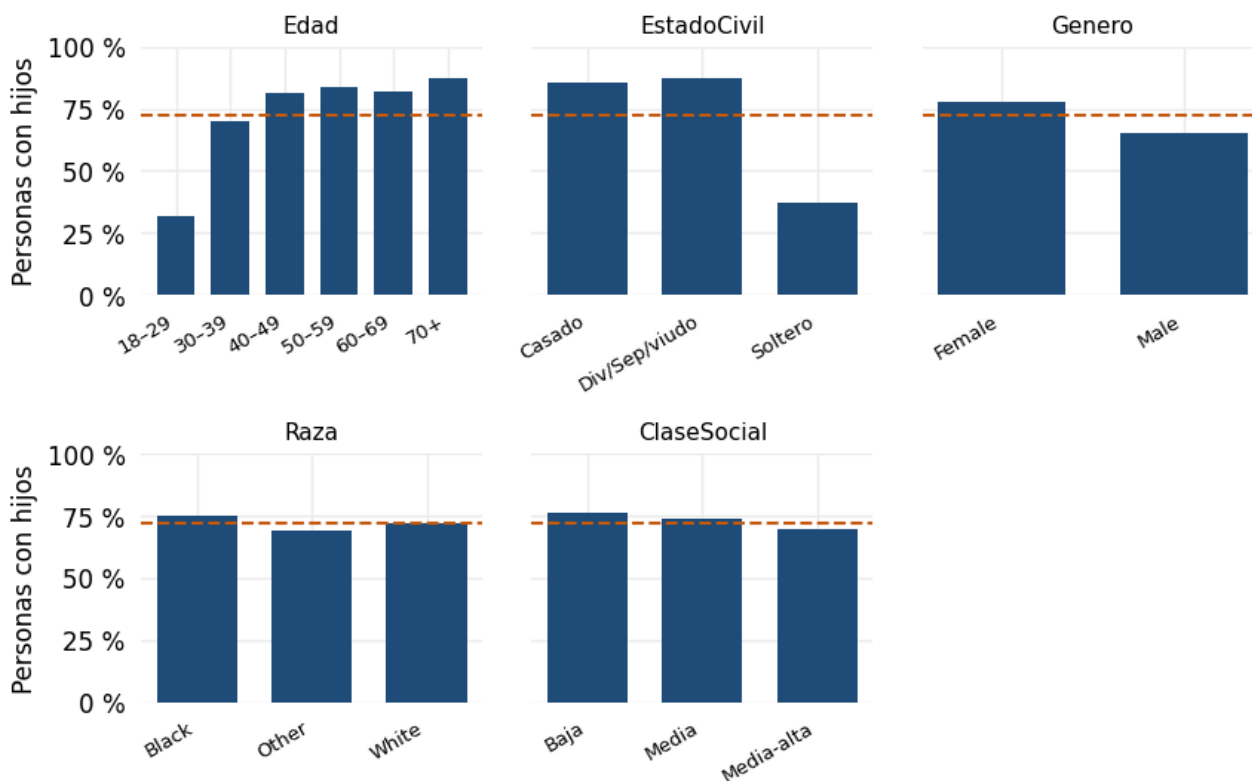
grupos_edad = pd.cut(datos["Edad"], [17, 29, 39, 49, 59, 69, np.inf],
                      labels=["18-29", "30-39", "40-49", "50-59", "60-69", "70+"])
variables_eda = {"Edad": grupos_edad, "EstadoCivil": datos["EstadoCivil"], "Genero": datos["Genero"],
                 "Raza": datos["Raza"], "ClaseSocial": datos["ClaseSocial"]}
tasa_global = np.mean(datos["Hijos"] == "Si")

fig, ejes = plt.subplots(2, 3, figsize=(8, 6), sharey=True)
for ax, (nombre, serie) in zip(ejes.flat, variables_eda.items()):
    prop = (datos["Hijos"] == "Si").groupby(serie.astype(str)).mean().sort_index()
    ax.bar(prop.index, prop.values, color=COL_PRINCIPAL, width=0.7)
    ax.axhline(tasa_global, color=COL_ACENTO, ls="--", lw=1.4)
    ax.set_title(nombre, fontsize=10, fontweight="normal", loc="center")
    ax.set_ylim(0, 1)
    ax.yaxis.set_major_formatter(lambda v, _: f"{v * 100:.0f} %")
    ax.tick_params(axis="x", rotation=30, labelsiz=8.5)
    for etiqueta in ax.get_xticklabels():
        etiqueta.set_ha("right")
ejes.flat[-1].set_visible(False)
ejes[0, 0].set_ylabel("Personas con hijos"); ejes[1, 0].set_ylabel("Personas con hijos")
titulos(fig, "Proporción de personas con hijos según su perfil",
        "La línea discontinua marca la proporción global")
fig.tight_layout(rect=(0, 0, 1, 0.90))
plt.show()

```

Proporción de personas con hijos según su perfil

La línea discontinua marca la proporción global



La edad y el estado civil muestran las diferencias más marcadas: la proporción de personas con hijos crece con la edad, y las personas solteras tienen hijos con mucha menor frecuencia que las casadas o las que alguna vez lo estuvieron. El género y la raza muestran diferencias menores.

3. Partición entrenamiento-prueba

Se reserva el 80 % de las observaciones para entrenamiento, estratificando por la variable objetivo. Se usa la réplica de `caret::createDataPartition()` con el generador de R (`set.seed(12345)`), por lo que la partición es idéntica a la de la versión en R.

```
trainIndex = create_data_partition(datos["Hijos"], p=0.8, rng=RRandom(12345))
data_train = datos.iloc[trainIndex - 1].reset_index(drop=True)
data_test = datos.drop(index=datos.index[trainIndex - 1]).reset_index(drop=True)
data_train["y"] = (data_train["Hijos"] == "Si").astype(int) # respuesta 0/1 para statsmodels
data_test["y"] = (data_test["Hijos"] == "Si").astype(int)

(tabla(pd.DataFrame({"Conjunto": ["Entrenamiento", "Prueba"],
                    "Observaciones": [len(data_train), len(data_test)],
                    "Con hijos": [data_train["y"].mean(), data_test["y"].mean()]}),
      "Resumen de la partición 80/20", "Semilla 12345")
  .fmt_percent(columns="Con hijos", decimals=1))
```

Resumen de la partición 80/20

Semilla 12345

Conjunto	Observaciones	Con hijos
Entrenamiento	1629	72.1%
Prueba	406	72.2%

4. Selección automática de variables

Se parte de dos modelos de referencia: el modelo nulo (solo intercepto) y el modelo completo (las 12 variables predictoras). Entre ellos, `step_glm()` (réplica de `step()`) busca el mejor subconjunto de variables con tres estrategias (*stepwise*, *forward* y *backward*) y dos criterios de información:

- **AIC** (criterio de Akaike): penaliza cada parámetro adicional con 2 unidades.
- **BIC** (criterio bayesiano): penaliza cada parámetro con $\log(n)$ unidades, lo que favorece modelos más simples.

```
predictores = [c for c in datos.columns if c != "Hijos"]
k_bic = math.log(len(data_train))

terminos = {
    "Stepwise · BIC": step_glm("y", data_train, [], predictores, "both", k_bic),
    "Forward · BIC": step_glm("y", data_train, [], predictores, "forward", k_bic),
    "Backward · BIC": step_glm("y", data_train, predictores, predictores, "backward", k_bic),
    "Stepwise · AIC": step_glm("y", data_train, [], predictores, "both", 2),
    "Forward · AIC": step_glm("y", data_train, [], predictores, "forward", 2),
    "Backward · AIC": step_glm("y", data_train, predictores, predictores, "backward", 2),
}

def ajustar_logistico(terms, datos_ajuste=data_train):
    return smf.glm("y ~ " + " + ".join(terms), data=datos_ajuste, family=sm.families.Binomial()).fit()

def n_parametros(modelo):
    return int(modelo.df_model) + 1
```

```
seleccionados = {nombre: ajustar_logistico(t) for nombre, t in terminos.items()}
modeloStepBIC, modeloStepAIC = seleccionados["Stepwise · BIC"], seleccionados["Stepwise · AIC"]
terms_bic, terms_aic = terminos["Stepwise · BIC"], terminos["Stepwise · AIC"]
```

```
resumen_seleccion = pd.DataFrame({
    "Método": list(seleccionados),
    "Parámetros": [n_parametros(m) for m in seleccionados.values()],
    "Variables": ["", " ".join(sorted(t)) for t in terminos.values()],
    "AIC": [m.deviance + 2 * n_parametros(m) for m in seleccionados.values()],
    "BIC": [m.deviance + k_bic * n_parametros(m) for m in seleccionados.values()]})
(tabla(resumen_seleccion, "Modelos obtenidos por selección automática", "Variables en orden alfabético")
 .fmt_number(columns=["AIC", "BIC"], decimals=1))
```

Modelos obtenidos por selección automática

Variables en orden alfabético

Método	Parámetros	Variables	AIC	BIC
Stepwise · BIC	9	ClaseSocial, Edad, EstadoCivil, Genero, Raza	1,437.0	1,485.5
Forward · BIC	9	ClaseSocial, Edad, EstadoCivil, Genero, Raza	1,437.0	1,485.5
Backward · BIC	9	ClaseSocial, Edad, EstadoCivil, Genero, Raza	1,437.0	1,485.5
Stepwise · AIC	19	ClaseSocial, Edad, Empleo, EstadoCivil, Genero, Politica, Raza, Region, Tamano	1,413.8	1,516.3
Forward · AIC	19	ClaseSocial, Edad, Empleo, EstadoCivil, Genero, Politica, Raza, Region, Tamano	1,413.8	1,516.3
Backward · AIC	19	ClaseSocial, Edad, Empleo, EstadoCivil, Genero, Politica, Raza, Region, Tamano	1,413.8	1,516.3

Las seis combinaciones producen **solo 2 modelos distintos**: las tres estrategias de búsqueda convergen a la misma solución para cada criterio. Las diferencias se deben exclusivamente al criterio:

- **BIC** selecciona 5 variables (**9 parámetros**): estado civil, edad, género, raza y clase social.
- **AIC**, con una penalización menor, incorpora además Empleo, Politica, Region, Tamano (**19 parámetros**). Las variables categóricas aportan un parámetro por cada nivel distinto del de referencia, lo que eleva rápidamente el número total.

Ninguno de los dos criterios selecciona el **signo del zodiaco**, que funciona como control: una variable sin relación plausible con la paternidad debe quedar excluida, y así ocurre.

5. Comparación por validación cruzada repetida

Se comparan tres candidatos: el modelo BIC, el modelo AIC y un **modelo de referencia basado en conocimiento del dominio**, con tres variables (estado civil, edad y raza). Cada modelo se evalúa con validación cruzada de 5 *folds* repetida 20 veces (100 estimaciones por métrica), usando la misma partición en todos. Los *folds* replican exactamente los de

`caret::train()` tras `set.seed(12345)`, incluida la semilla interna que `caret` extrae antes de generarlos.

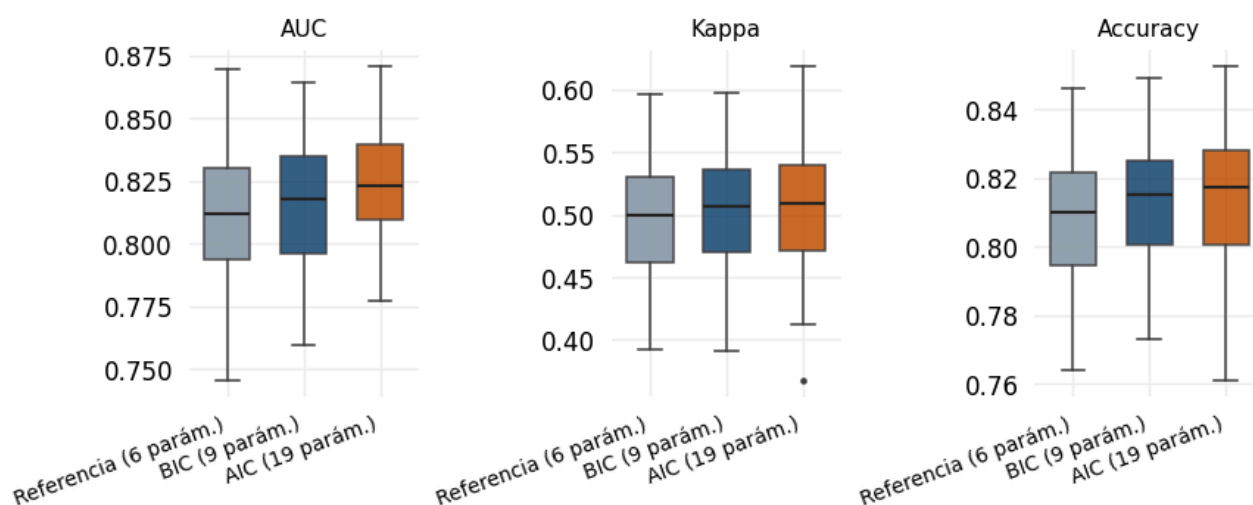
```
terms_manual = ["EstadoCivil", "Edad", "Raza"]
modeloManual = ajustar_logistico(terms_manual)
candidatos = {f"Referencia ({n_parametros(modeloManual)} parám.)": terms_manual,
              f"BIC ({n_parametros(modeloStepBIC)} parám.)": terms_bic,
              f"AIC ({n_parametros(modeloStepAIC)} parám.)": terms_aic}

folds_vc = caret_repeatedcv_folds(data_train["Hijos"], number=5, repeats=20, seed=12345)
filas = []
for nombre, terms in candidatos.items():
    for i, (idx_ent, idx_val) in enumerate(folds_vc):
        # GlmLogitR reproduce el IRLS de glm() de R (mismo criterio de parada), de modo que las
        # probabilidades coinciden con las de caret hasta la precisión de la máquina
        m = GlmLogitR("y ~ " + " + ".join(terms), data_train.iloc[idx_ent])
        val = data_train.iloc[idx_val]
        prob = m.predict(val)
        obs = val["y"].to_numpy() == 1
        pred = prob >= 0.5 # regla de clase de caret para glm
        # multiClassSummary promedia el AUC calculado para cada clase (con p y con 1 - p)
        auc_fold = (auc(obs, prob) + auc(~obs, 1 - prob)) / 2
        filas.append({"Modelo": nombre, "Resample": f"Fold{i % 5 + 1}.Rep{i // 5 + 1:02d}",
                     "AUC": auc_fold, "Kappa": kappa(obs, pred),
                     "Accuracy": np.mean(obs == pred)})
remuestreos = pd.DataFrame(filas)
```

```
metricas_vc = ["AUC", "Kappa", "Accuracy"]
fig, ejes = plt.subplots(1, 3, figsize=(8, 4.2))
for ax, met in zip(ejes, metricas_vc):
    datos_box = [remuestreos.loc[remuestreos["Modelo"] == m, met] for m in candidatos]
    cajas = ax.boxplot(datos_box, widths=0.6, patch_artist=True,
                      medianprops={"color": "#262626", "lw": 1.4},
                      whiskerprops={"color": "#404040"}, capprops={"color": "#404040"},
                      flierprops={"marker": "o", "markersize": 3, "markerfacecolor": "#404040",
                                   "markeredgecolor": "none"})
    for caja, color in zip(cajas["boxes"], PALETA_MODELOS):
        caja.set(facecolor=color, alpha=0.9, edgecolor="#404040")
    ax.set_xticks(range(1, 4), list(candidatos), rotation=20, ha="right", fontsize=9)
    ax.set_title(met, fontsize=10, fontweight="normal", loc="center")
titulos(fig, "Desempeño en validación cruzada repetida (5 folds × 20 repeticiones)",
        "Cada caja resume 100 estimaciones fuera de muestra")
fig.tight_layout(rect=(0, 0, 1, 0.88))
plt.show()
```

Desempeño en validación cruzada repetida (5 folds × 20 repeticiones)

Cada caja resume 100 estimaciones fuera de muestra



```
resumen_vc = (remuestreos.melt(id_vars=["Modelo", "Resample"], value_vars=metricas_vc,
                             var_name="Métrica", value_name="Valor")
              .groupby(["Métrica", "Modelo"], sort=False)["Valor"]
              .agg(Media="mean", **{"Desv. est.": "std"}, Mínimo="min", Mediana="median", Máximo="max")
              .reset_index()[["Modelo", "Métrica", "Media", "Desv. est.", "Mínimo", "Mediana", "Máximo"]])
(tabla(resumen_vc, "Resumen de la validación cruzada repetida")
 .fmt_number(columns=["Media", "Desv. est.", "Mínimo", "Mediana", "Máximo"], decimals=4))
```

Modelo	Métrica	Media	Desv. est.	Mínimo	Mediana	Máximo
Referencia (6 parám.)	AUC	0.8105	0.0246	0.7451	0.8117	0.8696
BIC (9 parám.)	AUC	0.8169	0.0250	0.7593	0.8178	0.8643
AIC (19 parám.)	AUC	0.8237	0.0220	0.7773	0.8230	0.8709
Referencia (6 parám.)	Kappa	0.4957	0.0446	0.3921	0.4992	0.5955
BIC (9 parám.)	Kappa	0.5004	0.0468	0.3908	0.5060	0.5973
AIC (19 parám.)	Kappa	0.5029	0.0469	0.3671	0.5086	0.6187
Referencia (6 parám.)	Accuracy	0.8081	0.0172	0.7638	0.8098	0.8462
BIC (9 parám.)	Accuracy	0.8125	0.0178	0.7730	0.8154	0.8492
AIC (19 parám.)	Accuracy	0.8145	0.0177	0.7607	0.8175	0.8528

El modelo AIC obtiene el mejor AUC medio (0.8237), seguido del modelo BIC (0.8169) y del de referencia (0.8105). Sin embargo, la ventaja del modelo AIC sobre el BIC (0.0067) es muy inferior a la variabilidad entre remuestreos (desviación estándar de 0.0250), y en Kappa y *accuracy* las diferencias son prácticamente nulas.

Siguiendo el **principio de parsimonia**, se elige el **modelo BIC**: logra un desempeño equivalente con 9 parámetros frente a 19, lo que lo hace más interpretable y menos propenso al sobreajuste.

6. Evaluación en el conjunto de prueba

```
umbral = data_train["y"].mean()
```

El umbral de clasificación por defecto (0.5) no es adecuado cuando las clases están desbalanceadas: clasificaría a casi todas las personas como "con hijos". Se usa como umbral la **proporción de personas con hijos en entrenamiento** (0.7213), que equilibra sensibilidad y especificidad.

```
probs_test = modeloStepBIC.predict(data_test).to_numpy()
pred_test = np.where(probs_test >= umbral, "Si", "No")
matriz = pd.crosstab(pd.Categorical(pred_test, ["No", "Si"]), data_test["Hijos"], dropna=False)
(tabla(pd.DataFrame({"Predicción": ["No", "Sí"], "Observado: No": matriz["No"].values,
                    "Observado: Sí": matriz["Si"].values}),
  "Matriz de confusión · Modelo BIC en prueba", "Columnas: clase observada"))
```

Matriz de confusión · Modelo BIC en prueba

Columnas: clase observada

Predicción	Observado: No	Observado: Sí
No	83	50
Sí	30	243

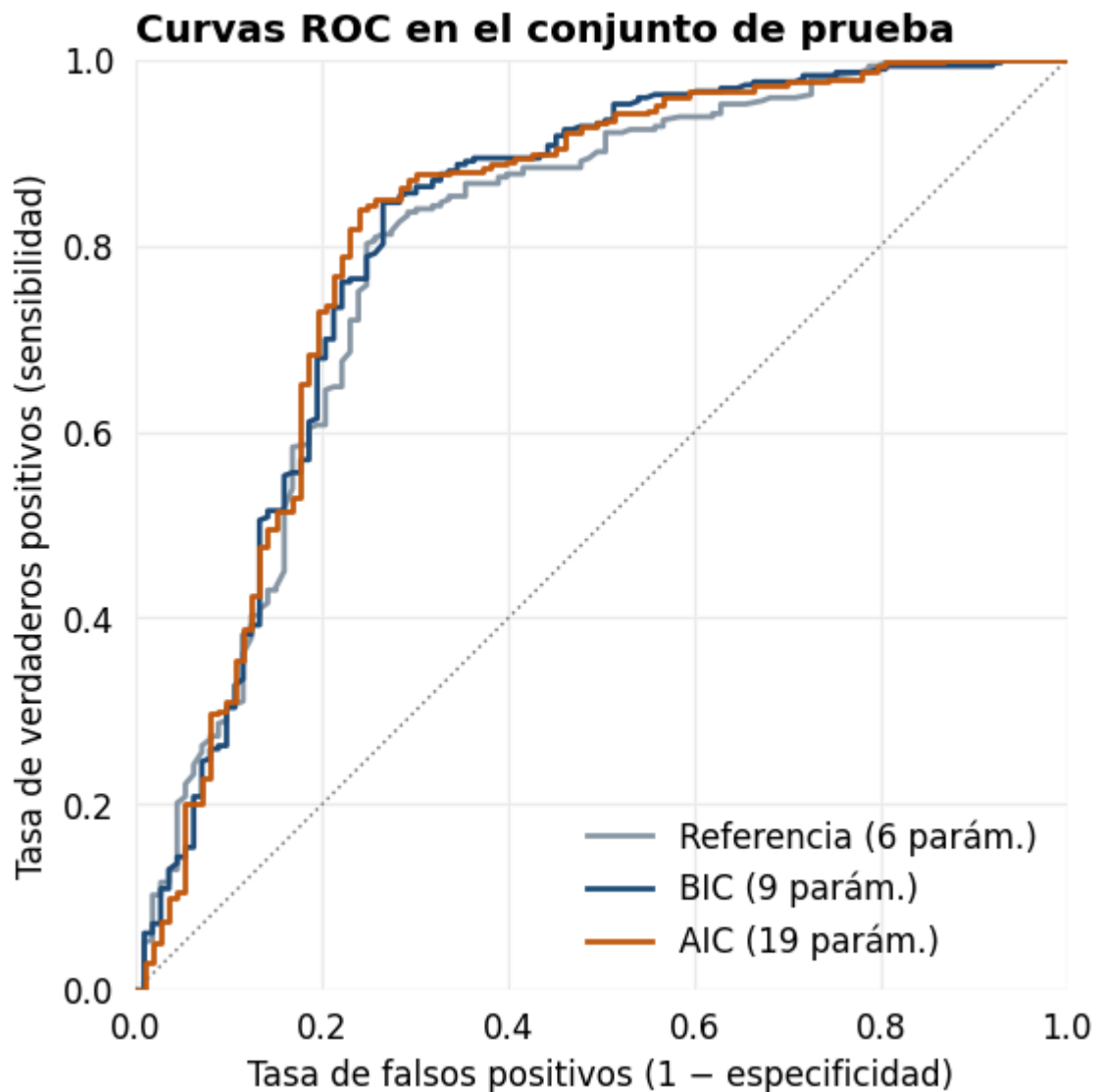
```
modelos_candidatos = {"Referencia": modeloManual, "BIC": modeloStepBIC, "AIC": modeloStepAIC}
prob_prueba = {nombre: modelos_candidatos[nombre.split(" ")[0]].predict(data_test).to_numpy()
                for nombre in candidatos}
metricas_prueba = pd.DataFrame([{"Modelo": n, **metricas_clasif(p, data_test["Hijos"], umbral)}
                                for n, p in prob_prueba.items()])
(tabla(metricas_prueba, "Métricas en el conjunto de prueba", f"Umbral de clasificación = {num(umbral, 4)}")
 .fmt_number(columns=list(metricas_prueba.columns[1:]), decimals=4)
 .tab_style(style=style.text(weight="bold"), locations=loc.body(rows=[1])))
```

Métricas en el conjunto de prueba

Umbral de clasificación = 0.7213

Modelo	AUC	Accuracy	Kappa	Sensibilidad	Especificidad
Referencia (6 parám.)	0.7994	0.7980	0.5157	0.8362	0.6991
BIC (9 parám.)	0.8130	0.8030	0.5348	0.8294	0.7345
AIC (19 parám.)	0.8157	0.8030	0.5442	0.8157	0.7699

```
fig, ax = plt.subplots(figsize=(5.6, 5.6))
ax.plot([0, 1], [0, 1], ls=":", color="grey", lw=1)
for (nombre, p), color in zip(prob_prueba.items(), PALETA_MODELOS):
    fpr, tpr = curva_roc(data_test["Hijos"], p)
    ax.plot(fpr, tpr, color=color, lw=1.8, label=nombre)
ax.set_aspect("equal"); ax.set_xlim(0, 1); ax.set_ylim(0, 1)
ax.legend(loc="lower right", frameon=False)
ax.set_title("Curvas ROC en el conjunto de prueba")
ax.set_xlabel("Tasa de falsos positivos (1 - especificidad)")
ax.set_ylabel("Tasa de verdaderos positivos (sensibilidad)")
plt.show()
```



En datos no vistos, el modelo BIC alcanza un **AUC de 0.813**. Clasifica correctamente al 80.3 % de las personas, con una sensibilidad del 82.9 % (personas con hijos identificadas) y una especificidad del 73.5 % (personas sin hijos identificadas). El índice Kappa de 0.535 indica un acuerdo moderado más allá del azar. Las curvas ROC de los tres modelos son casi indistinguibles, lo que confirma que el estado civil, la edad y la raza concentran la mayor parte de la información predictiva.

7. Interpretación del modelo seleccionado

Importancia de las variables (ANOVA de tipo II)

Para cada variable se compara el modelo BIC con el modelo que la excluye mediante una prueba de razón de verosimilitudes, que es lo que calcula `car::Anova(type = "II")` para modelos sin interacciones.

```
filas_anova = []
for t in terms_bic:
    reducido = ajustar_logistico([x for x in terms_bic if x != t])
    lr = reducido.deviance - modeloStepBIC.deviance
    gl = int(modeloStepBIC.df_model - reducido.df_model)
    filas_anova.append({"Variable": t, "Estadístico LR ( $\chi^2$ )": lr, "gl": gl, "p": stats.chi2.sf(lr, gl)})
anova2 = pd.DataFrame(filas_anova).sort_values("Estadístico LR ( $\chi^2$ )", ascending=False)
```

```
anova2["p-valor"], anova2["Sig."] = fmt_p(anova2["p"]), estrellas(anova2["p"])
(tabla(anova2.drop(columns="p"), "ANOVA de tipo II · Modelo BIC",
      "Prueba de razón de verosimilitudes por variable")
 .fmt_number(columns="Estadístico LR ( $\chi^2$ )", decimals=2)
 .cols_align("center", columns=["p-valor", "Sig."]))
```

ANOVA de tipo II · Modelo BIC

Prueba de razón de verosimilitudes por variable

Variable	Estadístico LR (χ^2)	gl	p-valor	Sig.
EstadoCivil	196.85	2	< 0.001	***
Edad	60.96	1	< 0.001	***
Genero	26.29	1	< 0.001	***
Raza	25.69	2	< 0.001	***
ClaseSocial	18.50	2	< 0.001	***

Las cinco variables son significativas. El **estado civil** es, con gran diferencia, la más importante, seguido de la edad. Una advertencia metodológica: como las variables se eligieron con los mismos datos, estos p-valores tienden a ser optimistas. Deben leerse como una medida de importancia relativa, no como pruebas de hipótesis formales.

Odds ratios

```
def nombres_r(modelo, terms, datos_ref=data_train):
    """Coeficientes con nombres y orden de R (p. ej. 'EstadoCivilSoltero')."""
    orden, mapa = [{"Intercept"}], {"Intercept": "(Intercept)"}
    for t in terms:
        if isinstance(datos_ref[t].dtype, pd.CategoricalDtype):
            for nivel in datos_ref[t].cat.categories[1:]:
                mapa[f"{t}[{nivel}]]"] = f"{t}{nivel}"
                orden.append(f"{t}{nivel}")
        else:
            mapa[t] = t
            orden.append(t)
    return mapa, orden

mapa, orden = nombres_r(modeloStepBIC, terms_bic)
cf = pd.DataFrame({"Coeficiente": modeloStepBIC.params, "Error estándar": modeloStepBIC.bse,
                  "p": modeloStepBIC.pvalues}).rename(index=mapa).loc[orden]
z = stats.norm.ppf(0.975)
odds = pd.DataFrame({
    "Término": cf.index, "Coeficiente": cf["Coeficiente"].values, "Error estándar": cf["Error estándar"].values,
    "Odds ratio": np.exp(cf["Coeficiente"]).values,
    "IC 95 % inf.": np.exp(cf["Coeficiente"] - z * cf["Error estándar"]).values,
    "IC 95 % sup.": np.exp(cf["Coeficiente"] + z * cf["Error estándar"]).values,
    "p-valor": fmt_p(cf["p"]), "Sig.": estrellas(cf["p"])}))
(tabla(odds, "Coeficientes y odds ratios · Modelo BIC",
      nota="Intervalos de confianza de Wald al 95 %. Significancia: *** p<0.001, ** p<0.01, * p<0.05, . p<0.1")
 .fmt_number(columns="Coeficiente", "Error estándar", decimals=4)
 .fmt_number(columns="Odds ratio", "IC 95 % inf.", "IC 95 % sup.", decimals=3)
 .cols_align("center", columns=["p-valor", "Sig."]))
```

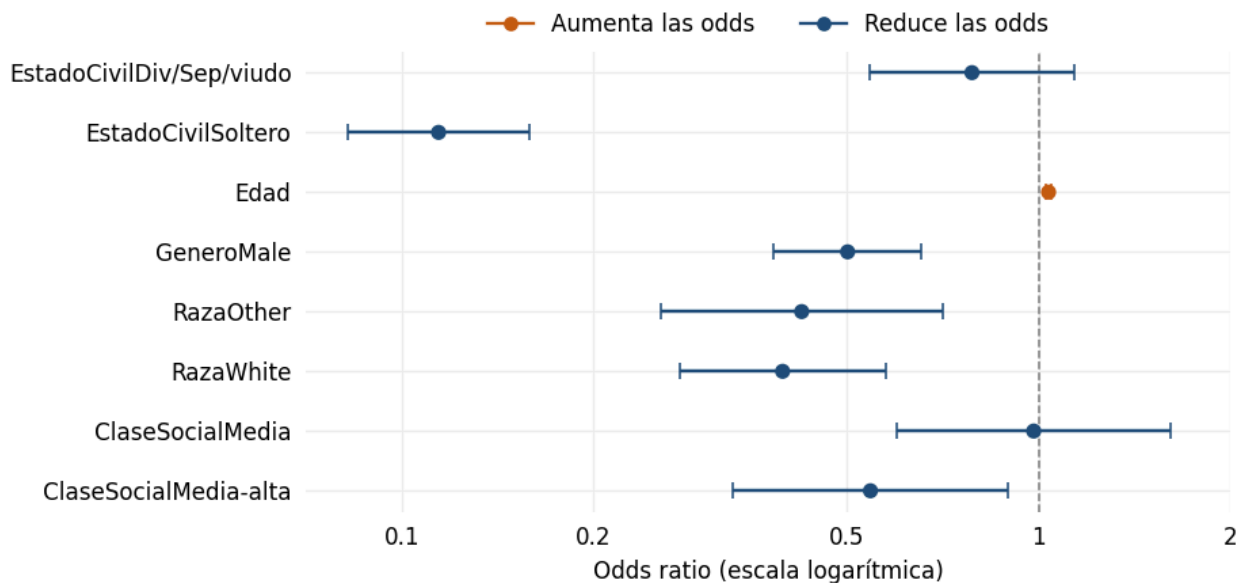
Coeficientes y odds ratios · Modelo BIC

Término	Coeficiente	Error estándar	Odds ratio	IC 95 % inf.	IC 95 % sup.	p-valor	Sig.
(Intercept)	1.5829	0.3848	4.869	2.290	10.350	< 0.001	***
EstadoCivilDiv/Sep/viudo	-0.2417	0.1883	0.785	0.543	1.136	0.1993	
EstadoCivilSoltero	-2.1694	0.1683	0.114	0.082	0.159	< 0.001	***
Edad	0.0351	0.0047	1.036	1.026	1.045	< 0.001	***
GeneroMale	-0.6911	0.1356	0.501	0.384	0.654	< 0.001	***
RazaOther	-0.8572	0.2601	0.424	0.255	0.707	< 0.001	***
RazaWhite	-0.9261	0.1904	0.396	0.273	0.575	< 0.001	***
ClaseSocialMedia	-0.0185	0.2525	0.982	0.598	1.610	0.9415	
ClaseSocialMedia-alta	-0.6086	0.2537	0.544	0.331	0.895	0.0164	*

Intervalos de confianza de Wald al 95 %. Significancia: *** p<0.001, ** p<0.01, * p<0.05, . p<0.1

```
f_or = odds[odds["Término"] != "(Intercept)"].iloc[:-1]
colores = [COL_ACENTO if v > 1 else COL_PRINCIPAL for v in f_or["Odds ratio"]]
fig, ax = plt.subplots(figsize=(8, 4))
ax.axvline(1, ls="--", color="grey", lw=1)
for i, (_, fila) in enumerate(f_or.iterrows()):
    ax.errorbar(fila["Odds ratio"], i,
                xerr=[fila["Odds ratio"] - fila["IC 95 % inf."], [fila["IC 95 % sup."] - fila["Odds ratio"]]],
                fmt="none", ecolor=colores[i], elinewidth=1.6, capsize=4)
ax.scatter(f_or["Odds ratio"], range(len(f_or)), color=colores, s=40, zorder=3)
ax.set_xscale("log")
ax.set_xticks([0.1, 0.2, 0.5, 1, 2], ["0.1", "0.2", "0.5", "1", "2"])
ax.minorticks_off()
ax.set_yticks(range(len(f_or)), f_or["Término"])
ax.legend(handles=[Line2D([], [], color=COL_ACENTO, marker="o", label="Aumenta las odds"),
                  Line2D([], [], color=COL_PRINCIPAL, marker="o", label="Reduce las odds")],
          loc="upper center", bbox_to_anchor=(0.5, 1.13), ncol=2, frameon=False)
ax.set_title("Odds ratios del modelo BIC con intervalos de confianza al 95 %", pad=34)
ax.set_xlabel("Odds ratio (escala logarítmica)")
plt.show()
```

Odds ratios del modelo BIC con intervalos de confianza al 95 %



Un *odds ratio* (OR) compara las *odds* (probabilidad de tener hijos dividida entre la probabilidad de no tenerlos) de una categoría con las de su categoría de referencia, **manteniendo constantes las demás variables**:

- **Estado civil.** Las *odds* de tener hijos de una persona soltera son 0.114 veces las de una persona casada, es decir, un 89 % menores. Las personas divorciadas, separadas o viudas no difieren significativamente de las casadas ($p = 0.199$), lo que refleja que la mayoría tuvo hijos durante su matrimonio.
- **Edad.** Cada año adicional multiplica las *odds* por 1.0358, un aumento del 3.6 % anual. Diez años más de edad equivalen a multiplicar las *odds* por 1.42.
- **Género.** Los hombres declaran tener hijos con *odds* un 50 % menores que las mujeres (OR = 0.501).
- **Raza.** Frente a las personas de raza negra (referencia), las *odds* son un 60 % menores para las personas blancas y un 58 % menores para otras razas.
- **Clase social.** Solo la clase media-alta se diferencia significativamente de la clase baja, con *odds* un 46 % menores.

8. Modelos regularizados

La regularización añade a la función de verosimilitud una penalización sobre el tamaño de los coeficientes, controlada por el parámetro λ . El parámetro α define el tipo de penalización:

- **Ridge ($\alpha = 0$):** penalización cuadrática. Reduce los coeficientes hacia cero, pero nunca los anula; útil ante predictores correlacionados.
- **LASSO ($\alpha = 1$):** penalización en valor absoluto. Anula coeficientes y, por tanto, **selecciona variables**.
- **Elastic Net ($0 < \alpha < 1$):** combina ambos efectos.

La clase `Glmnet` del módulo `rcompat` resuelve exactamente el mismo problema de optimización que `glmnet` de R: estandariza las variables internamente, usa la misma

secuencia de λ y las mismas reglas de parada, y devuelve los coeficientes en la escala original de las variables.

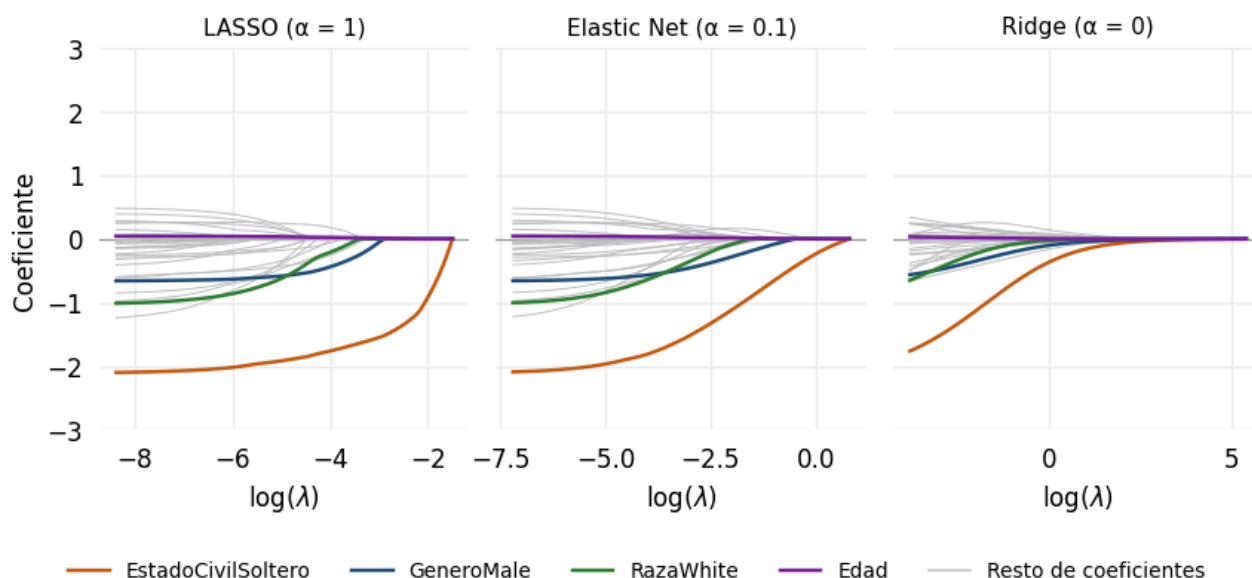
```
x = model.matrix(data_train.drop(columns="y"), "Hijos") # model.matrix(Hijos ~ ., data)[-1]
y = data_train["y"].to_numpy(float)
columnas_x = list(x.columns)

modeloLASSO = Glmnet(alpha=1).fit(x, y)
modeloRIDGE = Glmnet(alpha=0).fit(x, y)
modeloENET = Glmnet(alpha=0.1).fit(x, y)
```

```
destacados = {"EstadoCivilSoltero": COL_ACENTO, "GeneroMale": COL_PRINCIPAL, "RazaWhite": "#2E7D32",
              "Edad": "#7B1FA2"}
fig, ejes = plt.subplots(1, 3, figsize=(8, 4.6), sharey=True)
for ax, (nombre, m) in zip(ejes, {"LASSO ( $\alpha = 1$ )": modeloLASSO, "Elastic Net ( $\alpha = 0.1$ )": modeloENET,
                                   "Ridge ( $\alpha = 0$ )": modeloRIDGE}.items()):
    ax.axhline(0, color="#999999", lw=0.8)
    loglam = np.log(m.lambdas)
    for j, col in enumerate(columnas_x):
        if col not in destacados:
            ax.plot(loglam, m.beta[j], color="#BFBFBF", lw=0.6)
        for col, color in destacados.items():
            ax.plot(loglam, m.beta[columnas_x.index(col)], color=color, lw=1.6)
    ax.set_ylim(-3, 3)
    ax.set_title(nombre, fontsize=10, fontweight="normal", loc="center")
    ax.set_xlabel(r"$\log(\lambda)$")
ejes[0].set_ylabel("Coeficiente")
fig.legend(handles=[Line2D([], [], color=c, lw=1.6, label=n) for n, c in destacados.items()]
            + [Line2D([], [], color="#BFBFBF", lw=1, label="Resto de coeficientes")],
            loc="lower center", ncol=5, frameon=False, fontsize=9)
titulos(fig, "Trayectorias de los coeficientes según la penalización",
        "La penalización aumenta hacia la derecha")
fig.tight_layout(rect=(0, 0.07, 1, 0.88))
plt.show()
```

Trayectorias de los coeficientes según la penalización

La penalización aumenta hacia la derecha



En los tres casos, los coeficientes se contraen hacia cero a medida que aumenta la penalización, pero de forma distinta:

- **LASSO** anula los coeficientes de uno en uno. La mayoría desaparece pronto, y las últimas en salir son las más informativas (ser soltero, la edad y el género), lo que coincide con el análisis anterior.
- **Ridge** reduce todos los coeficientes de forma gradual y simultánea, sin anular ninguno. Por eso conserva siempre todos los parámetros.
- **Elastic Net** con $\alpha = 0.1$ muestra un comportamiento intermedio.

El coeficiente de la edad parece pequeño porque está expresado por año; su importancia es comparable a la de las variables categóricas una vez escalado.

9. Ajuste de hiperparámetros por validación cruzada

El valor de λ se elige por validación cruzada de 5 *folds*, maximizando el AUC. Para que la comparación entre distintos valores de α sea justa, **todos los modelos se evalúan con la misma asignación de observaciones a los folds**, generada con el generador de R
(`set.seed(12345)`).

```
foldid = np.array(RRandom(12345).sample([(i % 5) + 1 for i in range(len(y))])) # sample(rep(1:5, ...))
pd.Series(foldid).value_counts().sort_index()
```

```
1    326
2    326
3    326
4    326
5    325
Name: count, dtype: int64
```

LASSO: elección de λ

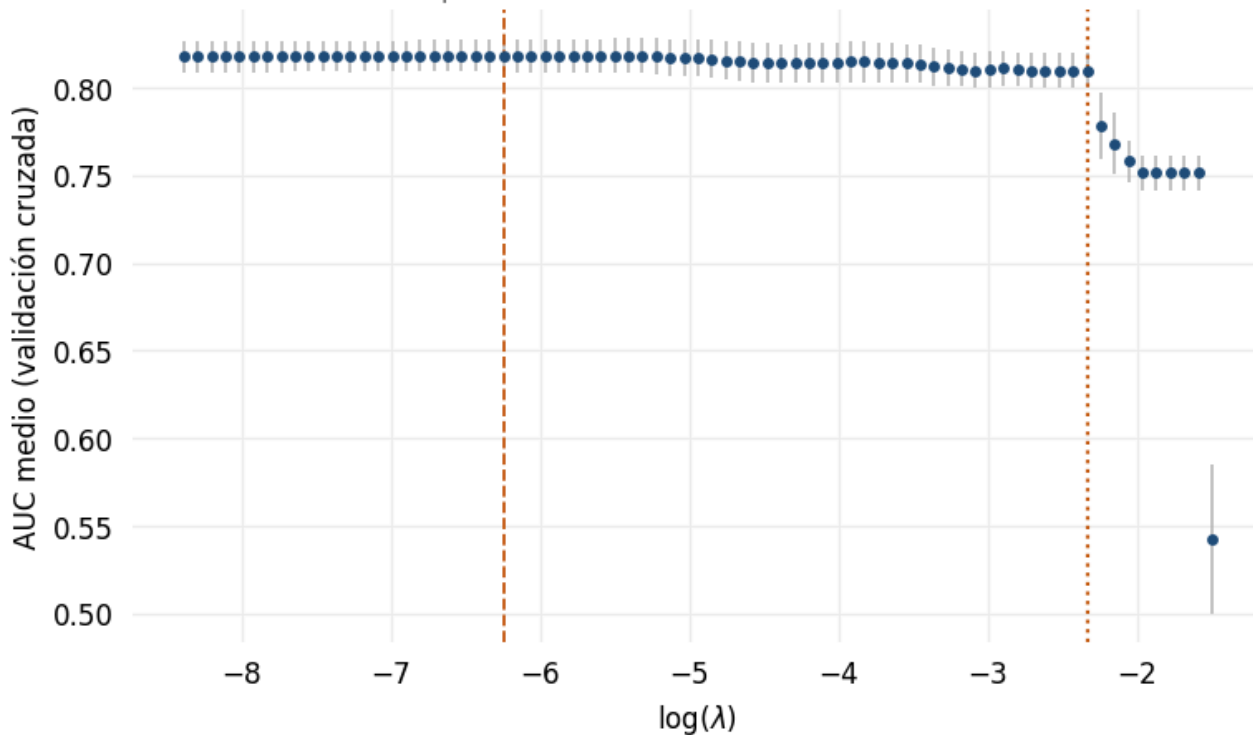
```
cv_lasso = CVGlmnet(alpha=1, foldid=foldid).fit(x, y)
idx = [cv_lasso.index_min, cv_lasso.index_lse]
(tabla(pd.DataFrame({"Criterio": ["λ mínimo (máximo AUC)", "λ 1-SE (regla de un error estándar)",
                                "Lambda": cv_lasso.lambdas[idx], "Parámetros no nulos": cv_lasso.nzero[idx],
                                "AUC medio": cv_lasso.cvm[idx], "Error estándar": cv_lasso.cvsd[idx]}],
                    "LASSO · λ seleccionados por validación cruzada")
    .fmt_number(columns=["Lambda", "AUC medio", "Error estándar"], decimals=4))
```

LASSO · λ seleccionados por validación cruzada				
Criterio	Lambda	Parámetros no nulos	AUC medio	Error estándar
λ mínimo (máximo AUC)	0.0019	28	0.8185	0.0092
λ 1-SE (regla de un error estándar)	0.0966	2	0.8099	0.0097

```
fig, ax = plt.subplots()
loglam = np.log(cv_lasso.lambdas)
ax.errorbar(loglam, cv_lasso.cvm, yerr=cv_lasso.cvsd, fmt="none", ecolor="#B3B3B3", elinewidth=1)
ax.scatter(loglam, cv_lasso.cvm, color=COL_PRINCIPAL, s=12, zorder=3)
ax.axvline(np.log(cv_lasso.lambda_min), color=COL_ACENTO, ls="--", lw=1.2)
ax.axvline(np.log(cv_lasso.lambda_lse), color=COL_ACENTO, ls=":", lw=1.4)
titulos(ax, "LASSO · AUC de validación cruzada según λ",
        "Discontinua: λ mínimo · punteada: λ 1-SE · barras: ± 1 error estándar")
ax.set_xlabel(r"$\log(\lambda)$"); ax.set_ylabel("AUC medio (validación cruzada)")
plt.show()
```

LASSO · AUC de validación cruzada según λ

Discontinua: λ mínimo · punteada: λ 1-SE · barras: ± 1 error estándar



El AUC se mantiene prácticamente constante en un amplio rango de λ y solo cae con las penalizaciones más fuertes. La **regla de un error estándar (1-SE)** elige el λ más grande (el modelo más simple) cuyo AUC está a menos de un error estándar del máximo. Aquí reduce los parámetros no nulos de 28 a 2, con una pérdida de AUC de solo 0.0086.

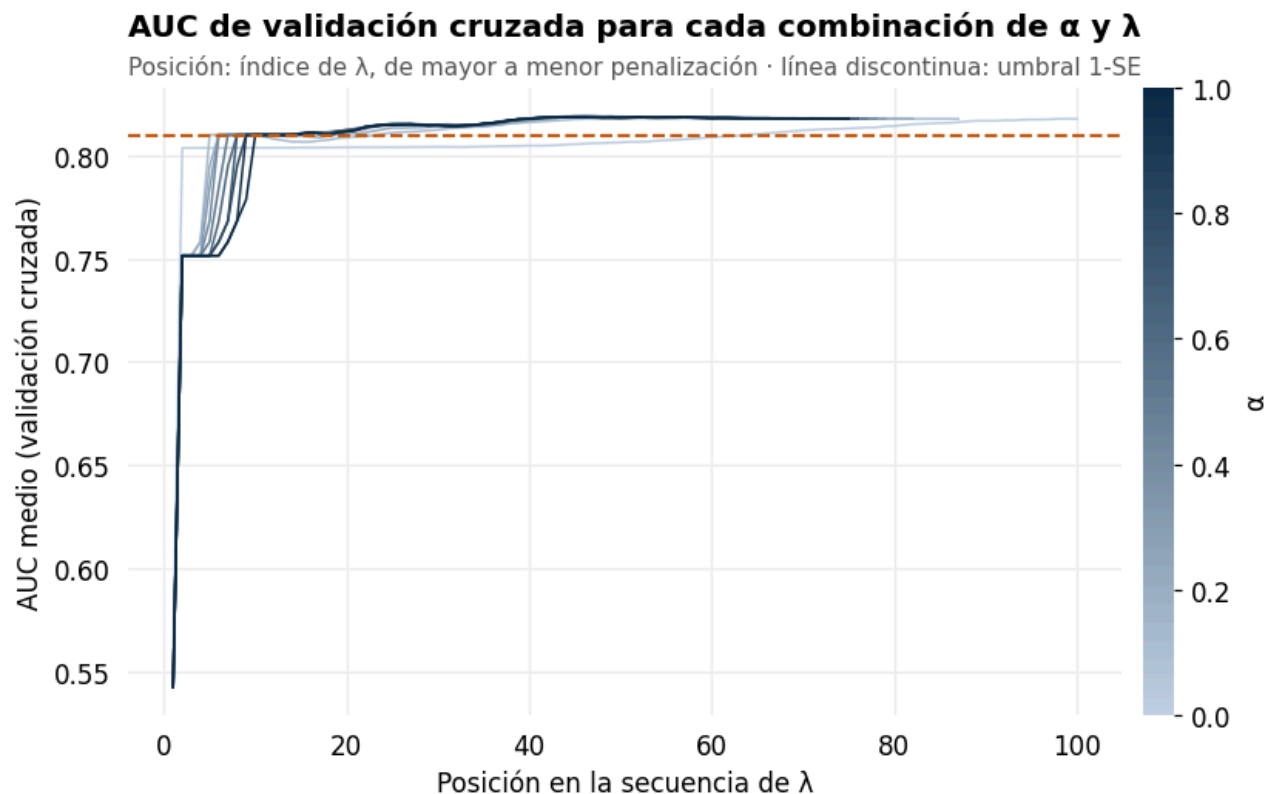
Búsqueda conjunta de α y λ

```
alphas = np.round(np.arange(0, 1.0001, 0.1), 1)
tunningEnet = [CVGlmnet(alpha=a, foldid=foldid).fit(x, y) for a in alphas]

resultado = pd.concat([pd.DataFrame({
    "alpha": a, "lambda": cv.lambdas, "numPar": cv.nzero, "auc_cv": cv.cvm, "auc_lo": cv.cvlo,
    "posicion": np.arange(1, len(cv.lambdas) + 1)} for a, cv in zip(alphas, tunningEnet)],
    ignore_index=True)

# Umbral de la regla 1-SE a partir del mejor modelo global
auc_lse = resultado.loc[resultado["auc_cv"].idxmax(), "auc_lo"]

fig, ax = plt.subplots(figsize=(8, 4.8))
cmap = mcolors.LinearSegmentedColormap.from_list("alpha", ["#BFD1E3", "#0B2A45"])
for a in alphas:
    sub = resultado[resultado["alpha"] == a]
    ax.plot(sub["posicion"], sub["auc_cv"], color=cmap(a), lw=1)
ax.axhline(auc_lse, color=COL_ACENTO, ls="--", lw=1.4)
barra = fig.colorbar(plt.cm.ScalarMappable(cmap=cmap, norm=mcolors.Normalize(0, 1)), ax=ax,
    fraction=0.035, pad=0.02)
barra.set_label("α"); barra.outline.set_visible(False)
titulos(ax, "AUC de validación cruzada para cada combinación de α y λ",
    "Posición: índice de λ, de mayor a menor penalización · línea discontinua: umbral 1-SE")
ax.set_xlabel("Posición en la secuencia de λ"); ax.set_ylabel("AUC medio (validación cruzada)")
plt.show()
```



La línea discontinua marca el **umbral de la regla 1-SE**: el AUC del mejor modelo menos su error estándar (0.8099). Cualquier combinación por encima de ella tiene un desempeño estadísticamente equivalente al óptimo. Salvo Ridge ($\alpha = 0$) y las penalizaciones más fuertes, casi todas las combinaciones superan el umbral, así que se elige la que **use menos parámetros**. La comparación se hace con cuatro decimales, porque diferencias de AUC menores a 0.0001 carecen de relevancia práctica y no deben decidir la selección. En caso de empate, se prefiere el mayor α (más selección de variables) y el mayor λ (más penalización).

```
# Se compara con 4 decimales: diferencias de AUC menores a 0.0001 no son relevantes
sobre_umbral = resultado[resultado["auc_cv"].round(4) >= round(auc_lse, 4)]
candidatos_lse = (sobre_umbral[sobre_umbral["numPar"] == sobre_umbral["numPar"].min()]
                  .sort_values(["alpha", "lambda"], ascending=False).reset_index(drop=True))
(tabla(candidatos_lse.head(8), "Combinaciones con el menor número de parámetros sobre el umbral 1-SE",
      "Se muestran las 8 primeras; la primera fila es la seleccionada")
 .fmt_number(columns=["lambda", "auc_cv", "auc_lo"], decimals=4)
 .cols_label(alpha="α", **{"lambda": "λ"}, numPar="Parámetros no nulos", auc_cv="AUC medio",
              auc_lo="AUC - 1 EE", posicion="Posición")
 .tab_style(style=style.text(weight="bold"), locations=loc.body(rows=[0])))
```

Combinaciones con el menor número de parámetros sobre el umbral 1-SE

Se muestran las 8 primeras; la primera fila es la seleccionada

α	λ	Parámetros no nulos	AUC medio	AUC – 1 EE	Posición
1.0	0.0966	2	0.8099	0.8002	10
1.0	0.0881	2	0.8099	0.8002	11
1.0	0.0802	2	0.8099	0.8002	12
1.0	0.0731	2	0.8099	0.8002	13
1.0	0.0666	2	0.8099	0.8002	14
1.0	0.0607	2	0.8102	0.8006	15
1.0	0.0553	2	0.8111	0.8012	16
0.9	0.1179	2	0.8099	0.8002	9

Modelo regularizado definitivo

```
alpha_opt = candidatos_lse.loc[0, "alpha"]
lambda_opt = candidatos_lse.loc[0, "lambda"]
modeloEnetDef = tuningEnet[int(np.where(alphas == alpha_opt)[0][0]).glmnet_fit
a0_def, beta_def = modeloEnetDef.coef_s(lambda_opt)
coef_def = pd.Series(np.concatenate([a0_def, beta_def[:, 0]]), index=["(Intercept)"] + columnas_x)

no_nulos = coef_def[coef_def != 0]
(tabla(pd.DataFrame({"Término": no_nulos.index, "Coeficiente": no_nulos.values,
                    "Odds ratio": np.exp(no_nulos.values)}),
        f"Coeficientes no nulos del modelo regularizado ( $\alpha = \{alpha\_opt:g\}$ ,  $\lambda = \{num(lambda\_opt, 4)\}$ ",
        nota="Coeficientes en la escala original de las variables, contraídos por la penalización.")
    .fmt_number(columns=["Coeficiente", "Odds ratio"], decimals=4))
```

Coeficientes no nulos del modelo regularizado ($\alpha = 1$, $\lambda = 0.0966$)

Término	Coeficiente	Odds ratio
(Intercept)	1.2402	3.4564
Edad	0.0029	1.0029
EstadoCivilSoltero	-1.2533	0.2855
Coeficientes en la escala original de las variables, contraídos por la penalización.		

El modelo definitivo, con $\alpha = 1$ y $\lambda = 0.0966$, conserva solo 2 variables con coeficiente no nulo: ser soltero y la edad. Coincide con el λ 1-SE del LASSO de la sección anterior. Los signos concuerdan con los del modelo logístico: ser soltero reduce la probabilidad de tener hijos y la edad la aumenta. Sus magnitudes son menores porque la penalización las contrae hacia cero. Este sesgo deliberado reduce la varianza del modelo, pero implica que los *odds ratios* no deben interpretarse con la misma literalidad que en el modelo sin penalizar.

10. Evaluación del modelo regularizado en prueba

```

x_test = model_matrix(data_test.drop(columns="y"), "Hijos")
probs_test_enet = modeloEnetDef.predict_proba_s(x_test, lambda_opt)[: , 0]
n_par_enet = int((coef_def != 0).sum())

comparacion_final = pd.DataFrame([
    {"Modelo": f"Logístico BIC ({n_parametros(modeloStepBIC)} parámetros)",
      **metricas_clasif(probs_test, data_test["Hijos"], umbral)},
    {"Modelo": f"Regularizado,  $\alpha$  = {alpha_opt:g} ({n_par_enet} parámetros)",
      **metricas_clasif(probs_test_enet, data_test["Hijos"], umbral)}])
(tabla(comparacion_final, "Modelo logístico vs. modelo regularizado en prueba",
      f"Umbral de clasificación = {num(umbral, 4)}")
.fmt_number(columns=list(comparacion_final.columns[1:]), decimals=4))

```

Modelo logístico vs. modelo regularizado en prueba

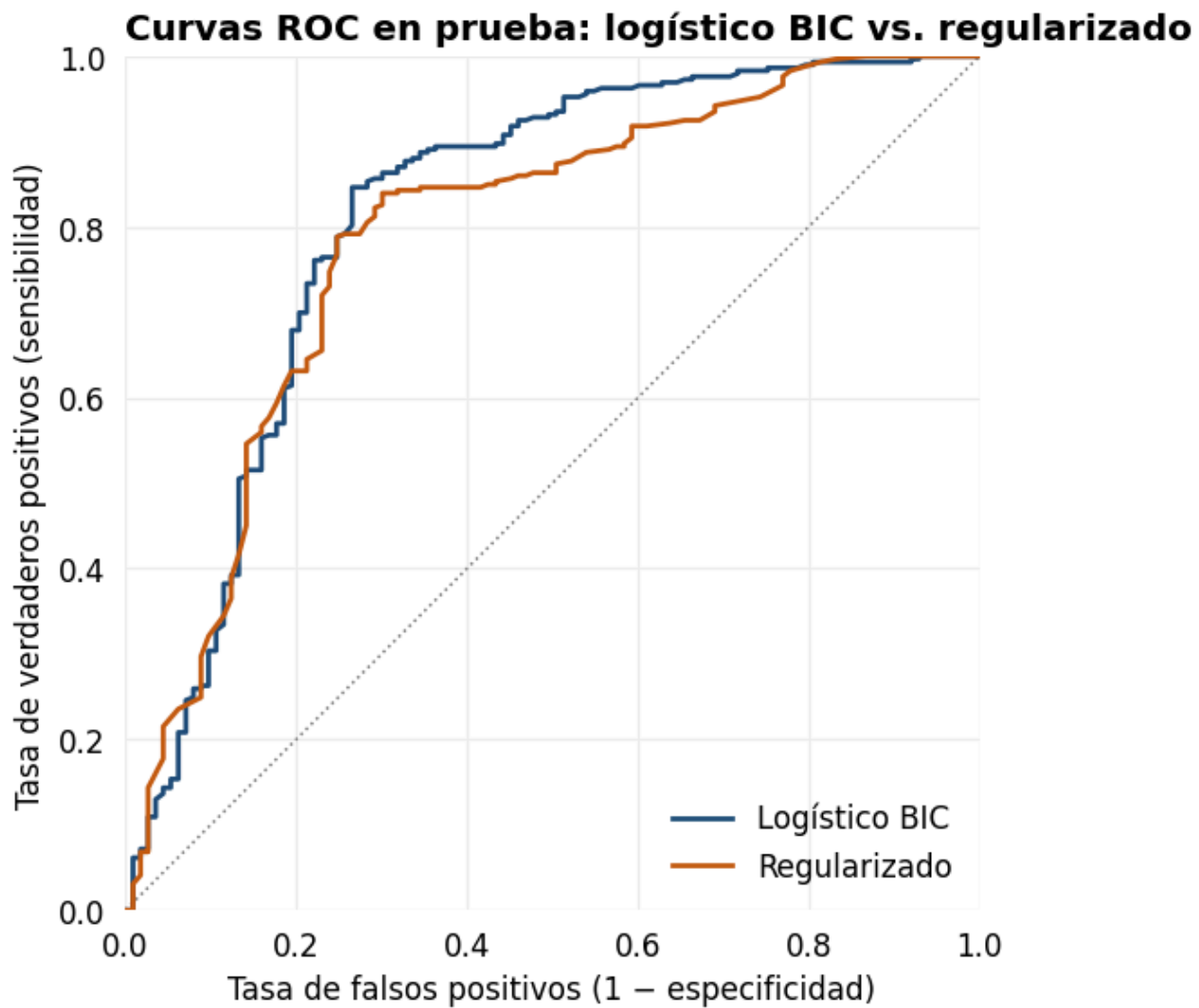
Umbral de clasificación = 0.7213

Modelo	AUC	Accuracy	Kappa	Sensibilidad	Especificidad
Logístico BIC (9 parámetros)	0.8130	0.8030	0.5348	0.8294	0.7345
Regularizado, α = 1 (3 parámetros)	0.7891	0.7956	0.5033	0.8430	0.6726

```

fig, ax = plt.subplots(figsize=(5.6, 5.6))
ax.plot([0, 1], [0, 1], ls=":", color="grey", lw=1)
for (nombre, p), color in zip({"Logístico BIC": probs_test, "Regularizado": probs_test_enet}.items(),
                               [COL_PRINCIPAL, COL_ACENTO]):
    fpr, tpr = curva_roc(data_test["Hijos"], p)
    ax.plot(fpr, tpr, color=color, lw=1.8, label=nombre)
ax.set_aspect("equal"); ax.set_xlim(0, 1); ax.set_ylim(0, 1)
ax.legend(loc="lower right", frameon=False)
ax.set_title("Curvas ROC en prueba: logístico BIC vs. regularizado")
ax.set_xlabel("Tasa de falsos positivos (1 - especificidad)")
ax.set_ylabel("Tasa de verdaderos positivos (sensibilidad)")
plt.show()

```



Con solo dos variables, el modelo regularizado obtiene un AUC de 0.789 frente a 0.813 del modelo BIC. Cede algo de capacidad discriminante a cambio de una simplicidad considerable: 3 parámetros frente a 9, y sin necesidad de recoger género, raza ni clase social.

11. Conclusiones

- **Qué determina la paternidad.** El estado civil es el factor dominante: las *odds* de tener hijos de una persona soltera son un 89 % menores que las de una casada. Le siguen la edad, el género, la raza y la clase social. Las variables de actitud (felicidad, política), la región y el signo del zodiaco no aportan información.
- **Modelo recomendado.** El modelo logístico seleccionado por BIC (9 parámetros) ofrece el mejor equilibrio entre precisión e interpretabilidad: AUC de 0.813 en prueba, sensibilidad del 82.9 % y especificidad del 73.5 %.
- **Alternativa mínima.** Si se requiere un modelo con la menor cantidad de información posible, el modelo regularizado con solo dos variables (estado civil y edad) mantiene un AUC de 0.789.
- **Solidez del proceso.** La selección de variables es estable (las tres estrategias coinciden), el control negativo (zodiaco) queda excluido y las métricas de validación cruzada y de prueba son coherentes entre sí.

Próximos pasos: explorar interacciones (por ejemplo, edad × estado civil) y efectos no lineales de la edad mediante *splines*; calibrar las probabilidades estimadas; y elegir el umbral de clasificación según el coste relativo de los falsos positivos y negativos en la aplicación concreta.

Anexo: entorno de ejecución

```
import sys, platform, matplotlib, scipy, statsmodels, great_tables
print("Python", sys.version.split()[0], ".", platform.platform())
for mod in (np, pd, statsmodels, scipy, matplotlib, pyreadr, great_tables):
    print(f"{mod.__name__:>14}: {mod.__version__}")
```

```
Python 3.12.3 · Linux-6.18.44-fc-v42-x86_64-with-glibc2.39
  numpy: 2.4.4
  pandas: 3.0.2
  statsmodels: 0.15.0
  scipy: 1.17.1
  matplotlib: 3.10.8
  pyreadr: 0.5.6
  great_tables: 1.0.0
```